

```
/**
 * server/lib/yt-dlp-backoff.ts
 *
 * Fix #4 – `3Dw4UB86S9g` Times Out Every 8 Hours Across Every Session
 *
 * PROBLEM: Specific video IDs that consistently time out keep hitting the
 * front of the queue at full priority every cycle:
 *   May 31 23:01 – 3Dw4UB86S9g timed out 480s – queued vault
 *   Jun 1  01:52 – 3Dw4UB86S9g timed out 480s – queued vault
 *   Jun 1  10:54 – 3Dw4UB86S9g timed out 480s – queued vault
 *   Jun 2  02:24 – 3Dw4UB86S9g timed out 480s – queued vault
 *
 * Each 480s timeout blocks the yt-dlp gate slot for 8 minutes, starving
 * other downloads. After 4 consecutive timeouts, it should wait 24 hours.
 *
 * SOLUTION: Escalating backoff tracker keyed by video ID + failure type.
 * Consecutive timeout failures for the same video ID get exponentially
 * longer cooldowns. A hard format error (not available) escalates faster.
 *
 * This complements the Resurrection Engine – the engine handles DB-level
 * status resets; this handles in-session retry scheduling.
 */
```

```
import { createLogger } from './logger.js';
import { storage }      from '../storage.js';
```

```
const log = createLogger('yt-dlp-backoff');
```

```
// ——— Backoff config —————
```

```
type FailureType = 'timeout' | 'format_unavailable' | 'network_error' | 'unknown';
```

```
const BACKOFF_SCHEDULE_MS: Record<FailureType, number[]> = {
  //           1st fail   2nd fail   3rd fail   4th fail   5th+ fail
  timeout:    [ 2 * 3600, 6 * 3600, 24 * 3600, 48 * 3600, 7 * 86400].map(s => s *
1000),
  format_unavailable: [24 * 3600, 72 * 3600, 7 * 86400, 14 * 86400, 30 * 86400].map(s => s *
1000),
  network_error:    [ 1 * 3600, 2 * 3600, 6 * 3600, 24 * 3600, 48 * 3600].map(s => s *
1000),
  unknown:          [ 4 * 3600, 12 * 3600, 48 * 3600, 72 * 3600, 7 * 86400].map(s => s *
1000),
};
```

```
// ——— In-memory failure tracking —————
```

```
// Keyed by youtubeId – survives within a session, resets on reboot
// For cross-reboot persistence, store in DB (see persistBackoff below)
```

```

interface BackoffEntry {
  youtubeId:    string;
  failureType:  FailureType;
  consecutiveFails: number;
  retryAfter:   Date;
  lastFailureAt: Date;
}

const backoffMap = new Map<string, BackoffEntry>();

// ——— Error classifier ———

export function classifyYtDlpError(errorMessage: string): FailureType {
  const msg = errorMessage.toLowerCase();
  if (msg.includes('timed out') || msg.includes('timeout'))
    return 'timeout';
  if (msg.includes('requested format is not available') || msg.includes('format'))
    return 'format_unavailable';
  if (msg.includes('network') || msg.includes('connection') || msg.includes('econnreset'))
    return 'network_error';
  return 'unknown';
}

// ——— Public API ———

/**
 * Call when a yt-dlp download fails for a video ID.
 * Returns the next retry time.
 */
export async function recordYtDlpFailure(
  youtubeId:    string,
  errorMessage: string,
): Promise<Date> {
  const failureType = classifyYtDlpError(errorMessage);
  const existing = backoffMap.get(youtubeId);

  const consecutiveFails = existing ? existing.consecutiveFails + 1 : 1;
  const scheduleIndex    = Math.min(consecutiveFails - 1,
BACKOFF_SCHEDULE_MS[failureType].length - 1);
  const backoffMs        = BACKOFF_SCHEDULE_MS[failureType][scheduleIndex];
  const retryAfter        = new Date(Date.now() + backoffMs);

  const entry: BackoffEntry = {
    youtubeId,
    failureType,
    consecutiveFails,
    retryAfter,
    lastFailureAt: new Date(),
  };
};

```

```

backoffMap.set(youtubeId, entry);

const backoffHours = Math.round(backoffMs / 1000 / 3600 * 10) / 10;
log.warn(
  `[YtDlpBackoff] ${youtubeId} failed (${failureType}) - ` +
  `consecutive fail #${consecutiveFails} - ` +
  `next retry in ${backoffHours}h at ${retryAfter.toISOString()}`
);

// Persist to DB so backoff survives reboots
await persistBackoff(youtubeId, entry);

return retryAfter;
}

/**
 * Check if a video ID is currently in backoff.
 * Call before attempting a yt-dlp download.
 *
 * Returns { blocked: true, retryAfter } if still in backoff.
 * Returns { blocked: false } if ready to retry.
 */
export async function isYtDlpBlocked(youtubeId: string): Promise<{
  blocked:    boolean;
  retryAfter: Date | null;
  reason:     string | null;
}> {
  // Check in-memory first
  const inMemory = backoffMap.get(youtubeId);
  if (inMemory && inMemory.retryAfter > new Date()) {
    return {
      blocked:    true,
      retryAfter: inMemory.retryAfter,
      reason:     `${inMemory.failureType} (fail #${inMemory.consecutiveFails})`,
    };
  }

  // Check DB for cross-reboot persistence
  const dbEntry = await loadBackoff(youtubeId);
  if (dbEntry && dbEntry.retryAfter > new Date()) {
    // Restore to in-memory cache
    backoffMap.set(youtubeId, dbEntry);
    return {
      blocked:    true,
      retryAfter: dbEntry.retryAfter,
      reason:     `${dbEntry.failureType} (fail #${dbEntry.consecutiveFails}, persisted)`,
    };
  }

  return { blocked: false, retryAfter: null, reason: null };
}

```

```

}

/**
 * Call when a download succeeds – clears the backoff for that video ID.
 */
export async function clearYtDlpBackoff(youtubeId: string): Promise<void> {
  if (backoffMap.has(youtubeId)) {
    backoffMap.delete(youtubeId);
    log.info(`[YtDlpBackoff] ${youtubeId} succeeded – backoff cleared`);
    await clearPersistedBackoff(youtubeId);
  }
}

// ——— DB persistence helpers ———
// Store backoff state in a simple key-value pattern using your existing storage

async function persistBackoff(youtubeId: string, entry: BackoffEntry): Promise<void> {
  try {
    await storage.setYtDlpBackoff(youtubeId, {
      failureType:      entry.failureType,
      consecutiveFails: entry.consecutiveFails,
      retryAfterIso:    entry.retryAfter.toISOString(),
      lastFailureIso:   entry.lastFailureAt.toISOString(),
    });
  } catch (err) {
    log.warn(`[YtDlpBackoff] Failed to persist backoff for ${youtubeId}:`, err);
  }
}

async function loadBackoff(youtubeId: string): Promise<BackoffEntry | null> {
  try {
    const raw = await storage.getYtDlpBackoff(youtubeId);
    if (!raw) return null;
    return {
      youtubeId,
      failureType:      raw.failureType as FailureType,
      consecutiveFails: raw.consecutiveFails,
      retryAfter:        new Date(raw.retryAfterIso),
      lastFailureAt:     new Date(raw.lastFailureIso),
    };
  } catch {
    return null;
  }
}

async function clearPersistedBackoff(youtubeId: string): Promise<void> {
  try {
    await storage.deleteYtDlpBackoff(youtubeId);
  } catch { /* silent */ }
}

```

```
// — Integration points —————  
//  
// In server/lib/yt-dlp-section-download.ts (or wherever downloads are spawned):  
//  
// BEFORE download:  
//   const blocked = await isYtDlpBlocked(youtubeId);  
//   if (blocked.blocked) {  
//     log.warn(`[YtDlpSection] ${youtubeId} in backoff until  
${blocked.retryAfter?.toISOString()} – skipping`);  
//     throw new Error(`yt-dlp backoff active for ${youtubeId}: ${blocked.reason}`);  
//   }  
//  
// AFTER timeout/failure:  
//   await recordYtDlpFailure(youtubeId, error.message);  
//  
// AFTER success:  
//   await clearYtDlpBackoff(youtubeId);  
//  
// Storage methods to add to server/storage.ts:  
//   setYtDlpBackoff(youtubeId: string, data: object): Promise<void>  
//   getYtDlpBackoff(youtubeId: string): Promise<any | null>  
//   deleteYtDlpBackoff(youtubeId: string): Promise<void>  
//   (store in a simple yt_dlp_backoff table: youtube_id, data jsonb, updated_at)
```